

Graph Neural Networks for Molecules

An introduction for undergraduate students

How to use these notes. Read Sections 1 to 3 before anything else, because the rest of the subject follows from the single rule stated in Section 3. Attempt each “Try it yourself” box as you reach it. The practice questions in Section 11 have full answers below them, so cover the answer before attempting the question. A glossary is provided at the end for quick reference.

1. The Problem We Are Trying to Solve

Suppose you want a computer to predict the solubility of a molecule.

The computer cannot look at a drawing of a molecule. It only understands numbers. So before anything else, you must turn the molecule into numbers.

The old way

A chemist picks a list of properties to calculate: molecular weight, number of rings, number of oxygen atoms, polar surface area, and so on. These are called descriptors. You compute perhaps twenty of them, feed the twenty numbers to a model, and hope they contain enough information.

Two problems arise:

1. Someone has to decide which descriptors to use, and different people pick different lists.
2. Information gets thrown away. Two different molecules can easily share the same molecular weight and the same number of rings.

The new way

Feed the molecule itself to the model, as a structure rather than as a summary. To do that, we write the molecule as a graph.

2. What Is a Graph?

A graph is just dots joined by lines.

- The dots are called nodes.
- The lines are called edges.

A molecule is already a graph. The atoms are the nodes and the bonds are the edges. You have been drawing graphs since your first organic chemistry class without calling them that.

To store a graph in a computer we need two tables.

2.1 Table 1: what each atom is (node features)

Number the atoms first. Then for each atom, write down which element it is, using a code where one column is 1 and the rest are 0. That code is called a one-hot vector.

Take methanol, CH_3OH . Number the atoms 1 to 6, where atoms 1, 2 and 3 are the methyl hydrogens, atom 4 is carbon, atom 5 is oxygen, and atom 6 is the hydroxyl hydrogen.

Atom	C	H	O
1	0	1	0
2	0	1	0
3	0	1	0
4	1	0	0
5	0	0	1
6	0	1	0

Read row 4: the 1 sits in the carbon column, so atom 4 is a carbon. As simple as that.

2.2 Table 2: which atoms are bonded (adjacency matrix)

Make a square table with one row and one column per atom. Put a 1 where two atoms are bonded and a 0 where they are not.

	1	2	3	4	5	6
1	0	0	0	1	0	0
2	0	0	0	1	0	0
3	0	0	0	1	0	0
4	1	1	1	0	1	0
5	0	0	0	1	0	1
6	0	0	0	0	1	0

Three things to notice:

- Rows 1, 2 and 3 each have a single 1, in column 4. Correct, because each methyl hydrogen is bonded only to the carbon.

- The table is symmetric. If atom 4 is bonded to atom 5, then atom 5 is bonded to atom 4. A chemical bond has no direction.
- The diagonal is all zeros, because no atom is bonded to itself.

Row 4 contains four 1s, matching carbon's valence of four. Checking that every row sum equals the valence of that atom is a quick way to catch mistakes.

Try it yourself. Write the two tables for water, H_2O . You should get a node table with 3 rows and a 3 by 3 adjacency matrix containing four 1s.

3. The One Big Rule

Here is the whole subject in one sentence.

The numbering of the atoms is arbitrary, so the answer must not depend on it.

Go back to methanol. Nothing stops you from calling the carbon atom 1 instead of atom 4. You would get a completely different-looking adjacency matrix, but the molecule has not changed and its solubility has not changed.

So any model we build must give the same answer for both numberings.

There are two versions of the rule, depending on what you are predicting.

What you predict	Name of the rule	What it means
A number for the whole molecule (solubility, energy)	Invariance	Renumber the atoms, and the answer stays exactly the same
A number for each atom (partial charge, NMR shift)	Equivariance	Renumber the atoms, and the answers get renumbered in the same way

Equivariance is the sensible default. If you swap atoms 2 and 5 in the input, you expect their partial charges to swap in the output. Nothing else should change.

3.1 How to check whether a formula obeys the rule

There is a one-line test, and it is worth memorising.

Look at the trainable numbers, meaning the weights. If any weight is attached to a particular atom, the formula is wrong.

Why? Because “atom number 3” is not a real thing. It is a label you invented when you numbered the molecule. If weight number 3 belongs to atom number 3, then renumbering shuffles the atoms without shuffling the weights, and everything breaks.

Weights are allowed to be attached to features, such as the carbon column or output channel 7. Weights are not allowed to be attached to positions in your numbering.

4. The Simplest Working Idea: Averaging Over Neighbours

Now we build an actual layer. We will build it in four small steps, and each step follows naturally from the one before.

Step 1. What should an atom's new description depend on? Its neighbours. In chemistry, an atom's behaviour depends on what it is bonded to. A carbon bonded to oxygen behaves differently from a carbon bonded to three hydrogens.

Step 2. How do we combine the neighbours? Take the average of their feature vectors. Why an average, rather than something like “first neighbour minus second neighbour”? Because an average does not care about order. Averaging 2, 4 and 6 gives 4, and so does averaging 6, 2 and 4. Order independence is exactly the rule from Section 3, obtained for free.

Step 3. What is wrong so far? Averaging has nothing to learn. There are no adjustable numbers, so the layer cannot be trained.

Step 4. Fix it. Multiply the neighbour features by a matrix of trainable weights before averaging. The weights are shared by every atom, so the test in Section 3 is passed.

That is the whole layer. It is called a graph convolution, and in the original paper by Kipf and Welling it is written like this:

$$v'_{il} = \sigma \left((1/d_i) \cdot e_{ij} \cdot v_{jk} \cdot w_{kl} \right)$$

The formula looks worse than it is. Read it as an instruction list.

Piece	Plain English
e_{ij}	Look at row i of the adjacency matrix, which picks out the neighbours of atom i and zeroes out everything else
v_{jk}	Take those neighbours' feature vectors

Piece	Plain English
w_{kl}	Multiply them by the trainable weights
$1 / d_i$	Divide by the number of neighbours, which turns the sum into an average
σ	Apply an activation function, exactly as in any ordinary neural layer
v'_{il}	The result, which is atom i's new feature vector

Notice that w has no atom index on it. It carries only k and l , which are feature numbers. The rule is satisfied.

4.1 One small fix: let an atom remember itself

Look carefully at the formula. Atom i 's new features are built entirely from its neighbours. Atom i 's own features do not appear, so the atom forgets what it was.

The standard trick is to pretend every atom is bonded to itself. In practice you add 1 to every diagonal entry of the adjacency matrix before you start. No change to the formula is needed. The zero diagonal from Section 2 was a fact about chemistry, and putting 1s there is a computational convenience, so the two statements do not conflict.

5. Why You Need More Than One Layer

After one graph convolution, each atom knows about its immediate neighbours.

Apply a second layer. Now each atom's neighbours have already absorbed information from their own neighbours, so after two layers each atom knows about everything two bonds away.

After L layers, each atom knows about everything within L bonds.

Trace it through methanol. At the start, the hydroxyl hydrogen (atom 6) knows only that it is a hydrogen. After one layer it knows it is a hydrogen attached to an oxygen. After two layers it knows it is a hydrogen attached to an oxygen that is attached to a carbon. Only at layer 2 does the carbon become visible to it.

Because chemistry involves environments several bonds wide, one layer is never enough. Typical models use three to six.

But do not overdo it. Averaging is a smoothing operation. Apply it too many times and every atom ends up with nearly the same feature vector, which is like stirring paint until the whole tin is one colour. The failure has a name, over-smoothing, and it means very deep stacks of graph convolutions often perform worse than shallow ones.

6. Getting an Answer Out: the Readout

Here is an awkward fact. A graph model takes a graph in and gives a graph out. But your label is usually not a graph.

6.1 Case A: one number per atom

Example: partial charge on each atom. Easy. Throw away the edges, keep the final atom feature vectors, and pass each one through the same small dense layer to get one number per atom.

6.2 Case B: one number for the whole molecule

Example: solubility. Now you have to squash many atom vectors into one molecule vector. The step is called the readout.

The order of operations matters, and there is only one correct order.

1. First combine all the atoms into a single vector, using something order-independent such as a sum or a mean.
2. Then pass that single vector through dense layers to get your prediction.

Doing it the other way round, meaning glue all the atom vectors end to end and then apply a dense layer, breaks the rule from Section 3. Gluing them end to end forces you to decide which atom goes first, and there is no correct answer to that.

6.3 Sum or mean? Ask whether the property is intensive or extensive

Straight out of your thermodynamics course, and it decides the readout.

Type	Does the value grow with system size?	Use	Examples
Intensive	No	mean	solubility, density, melting point, band gap
Extensive	Yes	sum	total energy, enthalpy of formation, mass, total charge

Think about why. Enthalpy of formation is roughly a sum of contributions from each atom, so a bigger molecule has a bigger value, and a sum reproduces that behaviour naturally. Solubility does not double when the molecule doubles in size, so a mean is right.

Get the choice wrong and the model fights the data forever. Using a mean for an extensive property means the model has to somehow reinvent the size dependence that you just averaged away.

Quick check. Heat capacity: extensive, use a sum. Refractive index: intensive, use a mean. Boiling point: intensive, use a mean.

7. A Different Way to Say the Same Thing: Message Passing

Everything above can be re-described in three steps, and the three-step version is how most papers talk.

Imagine each atom sending a note to its neighbours.

Step 1. Write the message. Each neighbour turns its feature vector into a message by multiplying by the weight matrix.

Step 2. Collect the messages. The receiving atom gathers all the messages arriving at it and combines them with something order-independent, usually a sum or a mean. The collecting step is often called pooling.

Step 3. Update. The atom uses the combined message to update its own feature vector.

A graph convolution is exactly this, with “multiply by a matrix” as the message, “take the mean” as the pooling, and “apply an activation” as the update.

The value of the three-step picture is that you can now invent new models by changing one step at a time. Almost every published architecture is one of these three steps swapped out. For example, replacing step 3 with a gated recurrent unit gives the gated graph model, and any architecture whose name begins with “gated” is doing exactly that.

A useful practical note: research has found that the choice in step 2 matters much less than people expect. Do not spend a week tuning your pooling function.

8. What About Molecules With No Bonds Drawn?

Sometimes your input is not a bond diagram. A snapshot from a molecular dynamics simulation, or a crystal structure, is just a list of atoms and their xyz coordinates. There is no bond table.

The model called SchNet handles exactly this case, and it is the standard first thing to try when predicting energies and forces. SchNet makes three decisions.

Nodes

Each element gets a trainable vector, looked up by atomic number.

Edges

Connect every atom to every other atom.

Students always object here. If everything is connected to everything, what was the point of using a graph at all? The answer is that we are not using a graph for its sparsity. We are using it because graph layers automatically obey the numbering rule from Section 3, and automatically handle a changing number of atoms. Feeding a raw coordinate list to an ordinary neural layer would fail on both counts.

Edge features

The information about geometry has to enter somewhere, and it enters through the edges. Each edge carries the distance r between the two atoms, converted into a vector like this:

$$e_k = \exp(-\gamma(r - \mu_k)^2)$$

That looks fancy, but the idea is simple. We want to one-hot encode the distance, but distance is a continuous number, so a true one-hot is impossible. Instead, put a row of Gaussian bumps at fixed distances μ_1, μ_2, μ_3 and so on, and record how strongly the actual distance lights up each bump.

Concretely, with bumps placed at 0, 2.5, 5, 7.5 and 10, a distance of 2 Å gives something like [0.02, 0.78, 0.00, 0.00, 0.00]. Most of the signal sits on the bump nearest to 2. The result is a smooth, blurry version of a one-hot vector.

One more detail worth knowing. SchNet does not use the usual ReLU activation. It uses a smoother one called shifted softplus. The reason is chemical: force is the derivative of energy with respect to position, so if you want forces out of your model, the model has to be smoothly differentiable. ReLU has a kink at zero and shifted softplus does not.

9. What Graph Models Cannot Do

Being honest about limits is part of understanding a method.

They can miss rings. A plain graph convolution cannot reliably tell a six-membered ring from two separate three-membered rings. In both cases every atom has exactly two neighbours, and every neighbourhood looks identical no matter how many layers you add. Adding depth does not help, because the problem is what the model can express, not how far information travels. The usual fix is to hand the model a ring-membership flag as an extra atom feature.

They ignore stereochemistry. Bond connectivity is a two-dimensional idea, and stereochemistry is fundamentally three-dimensional. Two enantiomers have identical graphs. Since one enantiomer of a drug can be active while the other is not, the gap is a serious practical problem and not just a technicality.

Bond order is sometimes discarded. Simple graph convolutions cannot use edge features, so bond order often gets dropped. Benzene and cyclohexane then differ only through their hydrogen counts.

Are graphs even necessary? A molecule can also be written as a SMILES string, and text models can read strings. For per-atom predictions you need a graph, because a string has no atoms to attach labels to. For whole-molecule properties such as solubility, string models sometimes win. The question is genuinely open, so do not let anyone tell you that graphs are always better.

10. Summary Card

Copy this into your notes.

1. A molecule is a graph. Atoms are nodes, bonds are edges.
2. Store it as a node table of one-hot element codes plus an adjacency matrix of 0s and 1s.
3. Atom numbering is arbitrary, so the model's answer must not depend on it. Check by looking for atom indices on the weights.

4. A graph convolution updates each atom by averaging its neighbours' features after multiplying by shared trainable weights.
5. Add 1s to the diagonal so that each atom also sees itself.
6. After L layers, an atom sees L bonds away. Too many layers causes over-smoothing.
7. To get a molecule-level answer, reduce over atoms first and apply dense layers second. Never the other way round.
8. Mean for intensive properties, sum for extensive properties.
9. Any graph model can be written as message, pooling, update.
10. For xyz coordinates with no bonds, use SchNet: connect everything, and put distance into the edges through Gaussian bumps.

11. Practice Questions

Question 1. Write the node table and adjacency matrix for ethanol, $\text{CH}_3\text{CH}_2\text{OH}$.

Answer. Nine atoms. Number them: 1 = methyl carbon, 2, 3 and 4 = its hydrogens, 5 = the other carbon, 6 and 7 = its hydrogens, 8 = oxygen, 9 = hydroxyl hydrogen.

Node table: put a 1 in the C column for atoms 1 and 5, in the O column for atom 8, and in the H column for atoms 2, 3, 4, 6, 7 and 9.

Bonds: 1–2, 1–3, 1–4, 1–5, 5–6, 5–7, 5–8, 8–9. Eight bonds, so the adjacency matrix has 16 ones in it, two per bond.

Check your work: rows 1 and 5 should each sum to 4, row 8 should sum to 2, and all the hydrogen rows should sum to 1.

Question 2. Which of these formulas obey the numbering rule? Use the test from Section 3.

$$(a) \quad v'_{ik} = \sigma \left((1/d_i) \cdot e_{ij} \cdot v_{jk} \cdot w_k \right)$$

$$(b) \quad v'_{ik} = \sigma \left((1/d_i) \cdot e_{ij} \cdot v_{jk} \cdot w_{jk} \right)$$

Answer. (a) Fine. The weight w_k carries only a feature index. Every atom is treated identically, so renumbering changes nothing. The formula is weaker than the real graph convolution, because it can only rescale each feature channel and cannot mix channels together, but it is legal.

(b) Broken. The weight w_{jk} carries j, which is a sending atom's number, so each atom would have its own private weights. Renumber the molecule and the atoms no longer line up with their weights, giving a different answer for the same molecule.

Question 3. You want to predict the total electronic energy of a molecule. Should your readout use a sum or a mean? What if you instead predict the energy per atom?

Answer. Total energy is extensive, so use a sum. A molecule with twice as many atoms should have roughly twice the energy, and a sum produces that naturally.

Energy per atom is intensive, so use a mean. Notice that the physical system did not change at all. Only the way you chose to write the target changed, and that alone flips the correct readout.

Question 4. A model has three graph convolution layers. Can it tell the difference between 1-chlorobutane and 1-chlorohexane?

Answer. Partly. Three layers means each atom sees three bonds out. The chlorine in both molecules sees $\text{Cl}-\text{C}-\text{C}$, which is identical, so the chlorine's own final feature vector is the same in both cases.

The molecules are still distinguishable overall, because the readout sums or averages over all atoms and the hexyl chain simply has more carbons contributing. A model predicting a per-atom property at the chlorine would fail, while a model predicting a whole-molecule property would succeed.

Question 5. Why does adding the identity matrix to the adjacency matrix matter? What goes wrong without it?

Answer. Without self-loops, an atom's updated features are built purely from its neighbours, so the atom completely forgets its own element identity after a single layer. A hydroxyl oxygen and an ether oxygen with the same neighbours would become indistinguishable.

There is a second, more practical failure. Dividing by the number of neighbours means dividing by zero for any isolated atom, such as a lone ion. Self-loops make the degree at least 1, so the division is always safe.

Question 6. In SchNet, what happens if the Gaussian bumps are placed too far apart?

Answer. Distances falling in the gaps between bumps light up almost nothing, so the edge feature vector is nearly all zeros and the geometric information is lost. Two chemically different distances can end up with the same near-empty encoding.

The opposite mistake also exists. Bumps that are too wide or too close together produce nearly identical vectors for distances that ought to be distinguished, and the model cannot see fine geometric differences. Spacing and width have to be chosen together.

12. Glossary

Term	Meaning
Node	A dot in a graph. For molecules, an atom.
Edge	A line in a graph. For molecules, a bond, or a distance in SchNet.

Term	Meaning
Adjacency matrix	Square table of 0s and 1s recording which nodes are joined.
One-hot vector	A vector of all 0s with a single 1, used to encode a category such as element.
Degree	The number of neighbours a node has. For an atom, its valence.
Invariant	Renumbering the atoms does not change the answer. Needed for whole-molecule properties.
Equivariant	Renumbering the atoms rennumbers the answers in the same way. Needed for per-atom properties.
Graph convolution	A layer that updates each atom by averaging its neighbours after multiplying by shared weights.
Self-loop	A pretend bond from an atom to itself, added so the atom does not forget its own identity.
Message passing	The three-step description of a graph layer: build messages, pool them, update the node.
Pooling	Combining incoming messages with an order-independent operation such as a sum or a mean.
Readout	Turning the model's graph output into the label you actually want.
Intensive	A property that does not scale with system size. Use a mean.
Extensive	A property that scales with system size. Use a sum.
Over-smoothing	The failure where too many layers make every atom's features nearly identical.
SchNet	A model that builds graphs from xyz coordinates by connecting all atoms and encoding distances.